

A TRAINING API FOR DISAGGREGATED RL

A Tinker-style training API, running on Modal.

Forward and backward as the only contract. You bring the algorithm; Modal brings the GPUs — and because the training server can live in your own environment, it can do full fine-tuning, not just LoRA. This note walks the architecture from a single server up to a multi-tenant, disaggregated-RL cloud.

SCROLL ↓

01 — THE THESIS

Find the boundary between algorithm and infrastructure.

Users want to change their application code without affecting the infrastructure underneath it. That means we need an API boundary between algorithm behavior and infrastructure — and once it exists, we can ship infrastructure improvements silently. Users never touch their application code to benefit.

It's the same move Modal already made for compute. The **Function** is the lowest-level API for general compute; the **Tinker interface** is the lowest-level API for training compute. Both abstract infrastructure away from application logic, and both are highly composable — higher-level products build on top.

GENERALIZED COMPUTE

Function

The lowest-level API for general compute.

APPLICATION

Your application code

API `@app.function()`

INFRASTRUCTURE

Containers · GPUs · scheduling · autoscaling

TRAINING COMPUTE

Tinker interface

The lowest-level API for training compute.

APPLICATION

Your training loop

API

`forward_backward · optim_step · sample`

INFRASTRUCTURE

Clusters · LoRA batching · fault recovery

Both abstract infrastructure away from application logic. Both are composable — the Gym, multi-tenant LoRA, and the disaggregated-RL cloud all build on top of this one boundary.



Silent infra upgrades

Ship scheduling, batching, and hardware wins beneath the boundary. Users never change application code to get them.



No redeploy

A new algorithm is just new code in your loop — not a fork of Slime or a custom algorithm function gated behind a full redeploy.



Built for multi-tenant LoRA

We need this exact boundary to batch many users' adapters onto shared GPUs later — the key to cost efficiency.

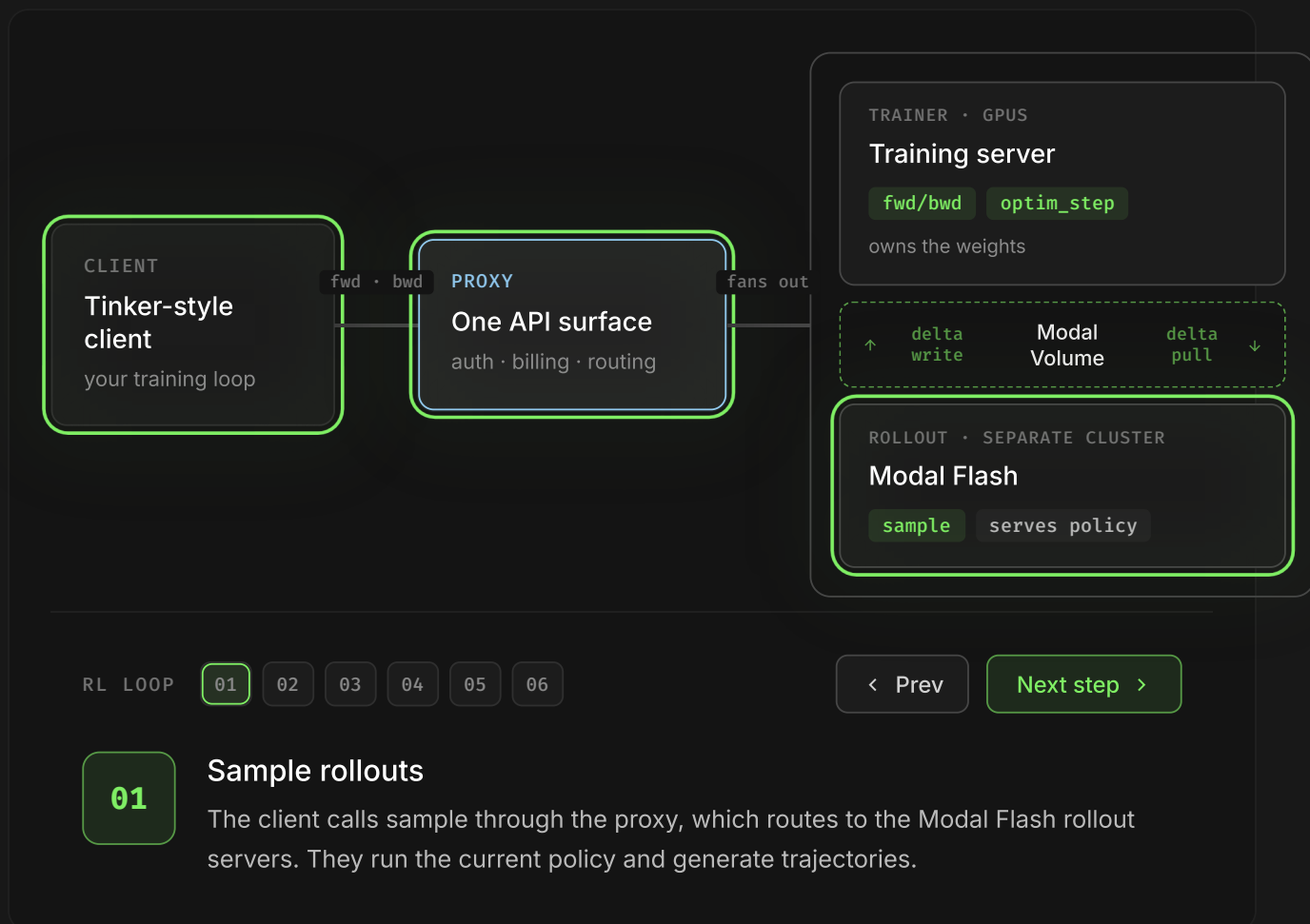
02 — THE DEFAULT SHAPE

Disaggregated RL: trainer GPUs $\neq$ rollout GPUs.

This is the shape everything else builds toward — and the reason a proxy sits in front from day one. Because we run disaggregated inference by default, a single client connection has to reach two different clusters at once, so the proxy is never optional. Two reasons for it: the minor one routes to the correct backend training server; the major one is that a **Modal Flash inference server** serves the rollouts.

The rollout GPUs are not the trainer GPUs and don't share a cluster. The trainer publishes weight updates as **delta-compressed** writes to a **Modal Volume**; rollout servers pull them. The proxy lets a single client connection hit both the trainer (`forward_backward`) and the rollout servers (`sample`)

through one API surface.



03 — THE ON-RAMP

The Training Gym's public surface barely changes.

The Gym is Modal's public front door for getting new users into RL. Today it wraps frameworks like Slime, or wraps one with Miles. A Tinker-style interface slots in as just another backend — the user-facing API stays put. What changes is the recipe behind it.



Everything is built on one training server.

A centralized process that accepts new LoRA-adapter initializations and runs forward / backward with a custom loss, plus the optimizer step. It can do LoRA or full fine-tuning. The client looks like Tinker — not the Modal client.

```
train.py

import modal_tinker as tinker

client = tinker.ServiceClient("https://train.acme.modal.run")

# one server ↔ one job — your environment, your GPUs
tc = client.create_training_client(
    base_model="Qwen/Qwen3-8B",
    tuning="lora",          # or "full" — full fine-tuning is on the table
    rank=32,
)

# write the algorithm; never touch the infra beneath it
fb      = tc.forward_backward(batch, loss_fn=my_custom_loss)
tc.optim_step(AdamParams(lr=1e-4))
sampler = tc.save_weights_and_get_sampling_client()
```

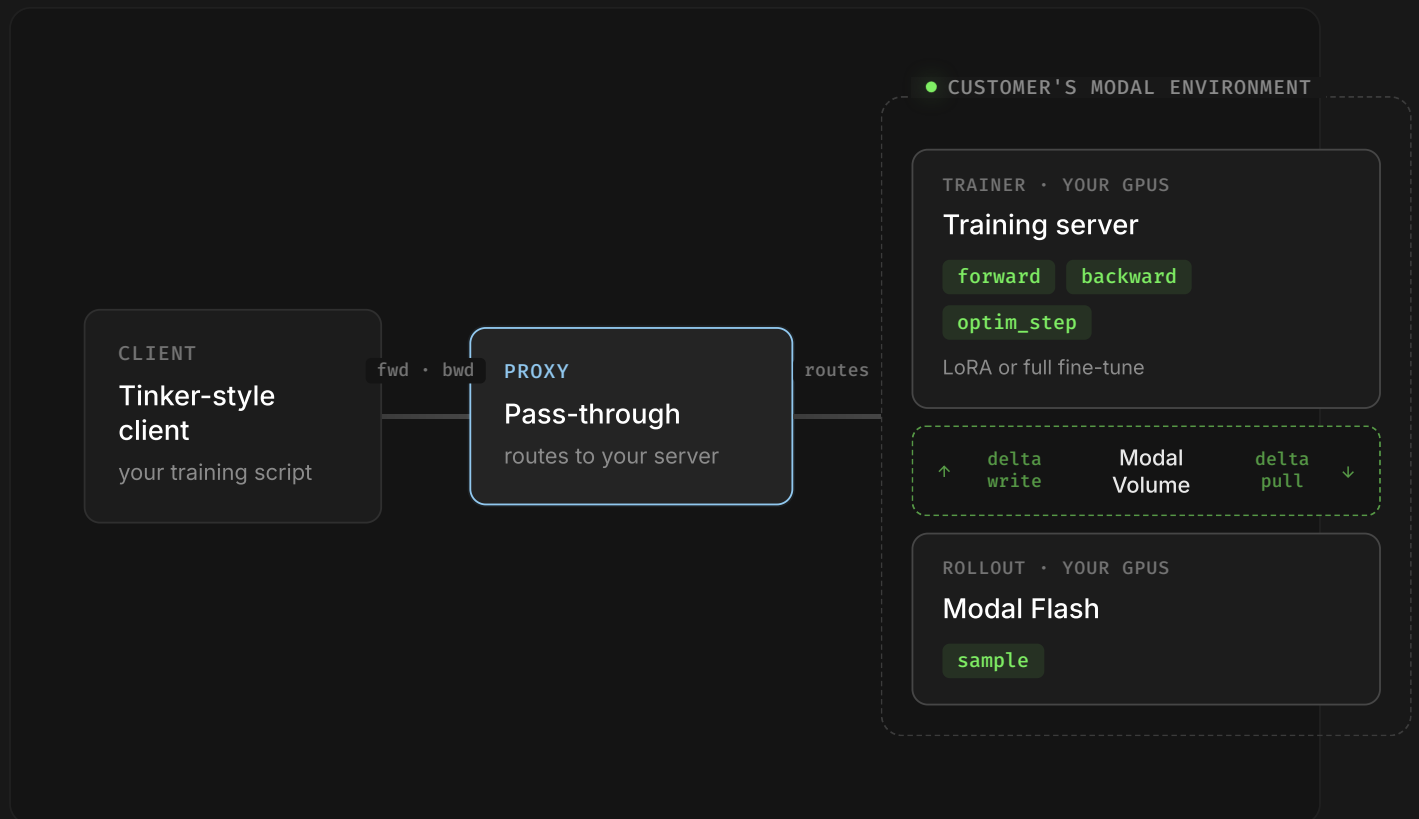
The custom `loss_fn` is what makes the seam expressive enough to cover SFT, DPO, and arbitrary RL objectives without a redeploy.

Singleton — one server, one job.

The first building block. The customer deploys the training server **single-tenant**, into **their own Modal environment**. One server corresponds to exactly one loaded LoRA — or one full-tune job. A proxy already sits in front — disaggregated inference is the default, so it's there from day one — but in the singleton case it's effectively pass-through.

Because it runs in **their** environment, full fine-tuning is genuinely on the table — an edge over Tinker, where shared multi-tenant compute makes full FT hard to offer.

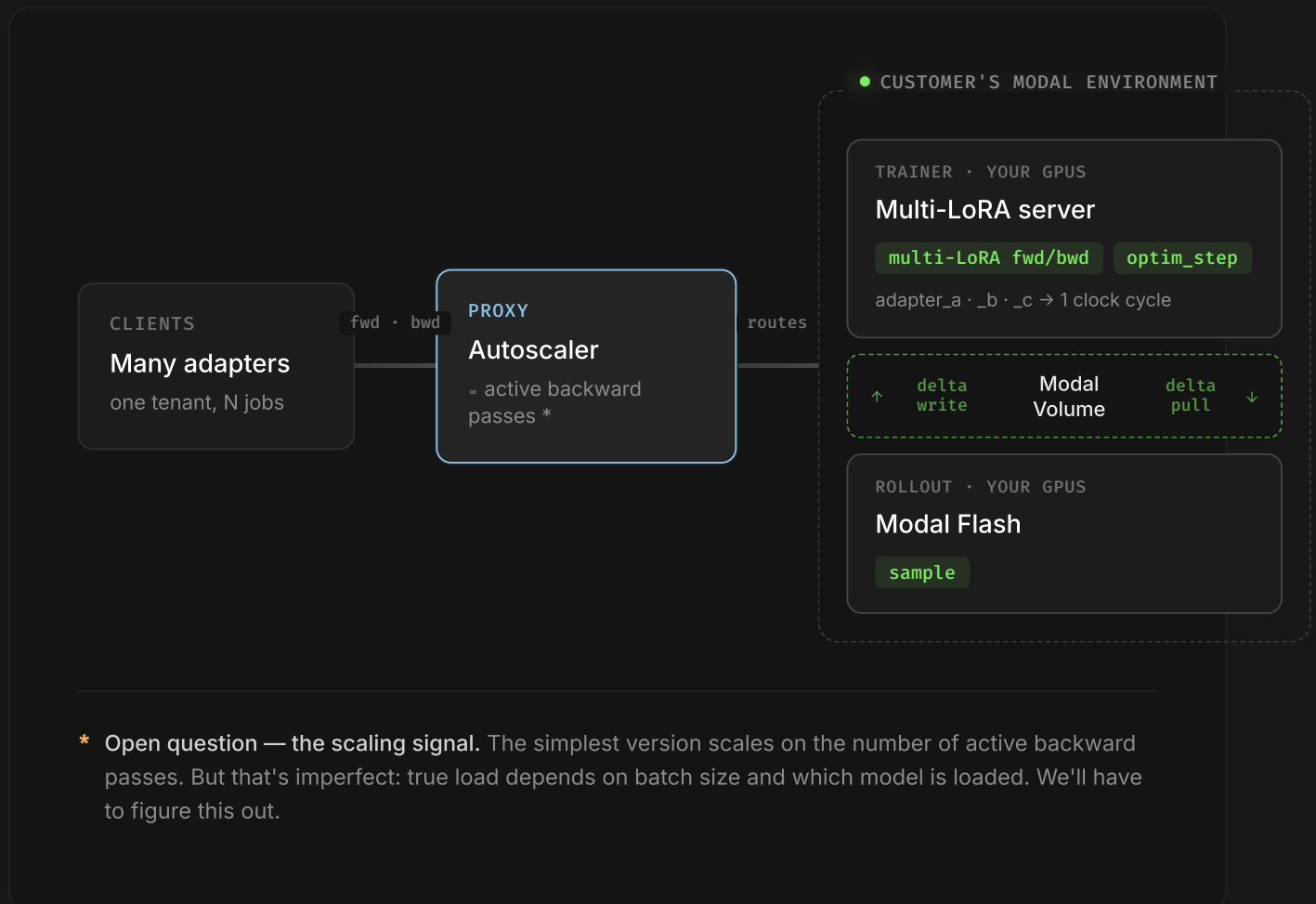
🕒 Click any node to inspect it.



Multi-LoRA — many adapters, still one tenant.

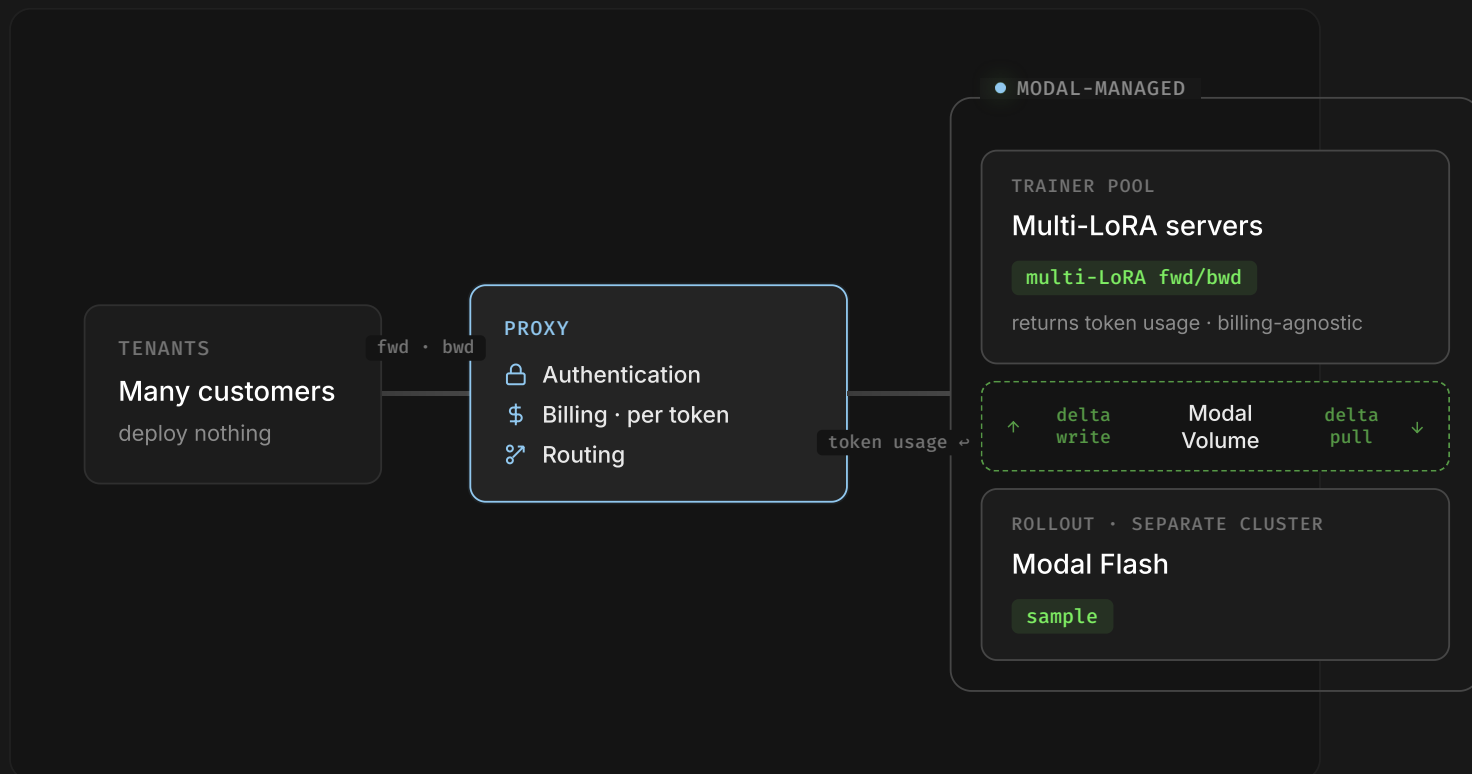
Still deployed in the customer's environment, but now one server hosts many adapters. We implement **multi-LoRA forward** and **multi-LoRA backward** so different adapters' requests batch together into a single clock cycle — the trick that makes shared compute economical.

The proxy that's been in front since day one now takes on autoscaling. The open question is the scaling signal.



Multi-tenant cloud — billing per token.

Once multi-LoRA is solid, we ship a fully managed offering: users deploy nothing. The proxy — the same one that has fronted the server since day one — now also owns **authentication and billing**. The server returns token usage; the proxy turns it into a bill. Keep billing concepts out of the training server entirely — they live only in the proxy.



THE PATH

Build the building blocks first.

PHASE 1

Singleton

One server $\mapsto$ one job, deployed into the customer's environment. LoRA or full FT. Tinker-style client.

>

PHASE 2

Multi-LoRA

Batched multi-LoRA forward/backward, single tenant. The proxy takes on autoscaling — signal still TBD.

>

PHASE 3

Multi-tenant cloud

Managed, bill-per-token. Proxy fronts trainer + Flash rollouts; weight sync over a Volume. Disaggregated RL.